

Spring 2026

Data Structures Section D20 — Lab 08

Friday May 08, 2026

Topic

Abstract Data Type: DoublyLinkedList

Recursion

Objective

- Understand the DoublyLinkedList functions implementations
- Recursive functions

Outcomes

1. Implement DoublyLinkedList Class Functions
2. Implement and use Recursive Functions

Content

The following sections will be covered during this lab session.

1. **Code and test** DoublyLinkedList Class Functions
2. **Code ad test Recursive Functions**

Tasks

Students are required to complete the following tasks in lab timings.

For the tasks 1-5, create a private repository in your account on Github: **S26_DSA_D11_Lab_8**. Upload codes with the names **Task_1**, **Task_2** and so on it.

Mandatory for Each Task

The following Steps must be **included** as **commented text** in **each .cpp file of each task**, before function main. **Missing these steps mean that you have NOT done the work yourself and it is from ChatGPT. (Please use Plain English or Urdu and No C++ Coding terms)**

Please note that these steps are more important than writing code, even at the expense of code incompleteness.

1. **Clearly and in details**, explain what you understand about the task at hand after carefully reading the given description and code.
2. Describe how will you do the task and the **solution** e.g. how to code a given functions or solution to the problem. You should use detailed steps and use the notations given in the slides for the LinkedList e.g. (A → B → C →) to show the progression of the solution.

3. Describe what **functions** you will **reuse** and **why**. Furthermore, explain the solution in terms of functions used. (**make sure your code is as per your solutions steps**). Show LinkedList e.g. (A → B → C →) to show the progression of the LinkedList after each function call.
4. Describe **your learning** from the task
5. Describe the **skills** you have developed and what type of problems you can code based on your experience of doing the task

Header files

The header files are provided as a zip file and these files must be used in your lab tasks

Task 1

Here you will write the code of class functions of the provided (DoublyLinkedList & ConcreteDoublyLinkedList) Classes and then test these functions in a suitable main

Instruction for the rest of the Tasks

In order to understand and develop expertise in writing recursive functions, you must do the following in comments before writing any code:

1. Decide the base case (e.g. smallest input)
2. Write at least 2 cases: 1) With only the smallest case 2) Nth case
3. Now write recursive class

Task 2

For the following iterative functions, write recursive versions and use a suitable main to test your functions:

```
void func(int n)
{
    for(i = n; i > 0; i--)
    {
        cout << i << endl;
    }
}
```

```
int sum(int n = 10)
{
    int total = 0;
    while(n > 0)
    {
        total += n;
        n--;
    }
    return total;
}
```

```
int max(int a[], int size)
{
    int m = a[0];
    for(int i = 1; i < size; i++)
    {
        if(m < a[i])
```

```

        {
            m = a[i];
        }
    }
    return m;
}

```

```

int countDigits(int n)
{
    int count = 0;
    while(n > 0)
    {
        if(n % 10 != 0)
        {
            count++;
        }
        n = n/10;
    }
    return count;
}

```

```

void displayDigits(int n)
{
    while(n > 0)
    {
        cout << n % 10 << " ";
        n = n/10;
    }
    cout << endl;
}

```

```

int binarySearch(int a[], int low, int high, int v)
{
    int mid = (high + low)/2;
    while(low <= high)
    {
        if(a[mid] == v)
        {
            return mid;
        }
        if(v > a[mid])
        {
            low = mid + 1;
        }
        if(v < a[mid])
        {
            high = mid - 1;
        }
        mid = (low + high)/2;
    }
    return -1;
}

```

```
}
```

Task 3

In this task, you will write recursive functions on ArrayStack. The iterative functions are given below:

```
void copyStack(ArrayStack<int> src, ArrayStack<int> & copy)
```

```
{
    if (src.isEmpty()) {
        return;
    }
    while(!src.isEmpty())
    {
        int v= 0;
        if(src.pop(v))
        {
            copy.push(v);
        }
    }
}
```

```
void display(ArrayStack<int> as)
```

```
{
    while(!as.isEmpty())
    {
        int v= 0;
        if(as.pop(v))
        {
            cout << v << " ";
        }
    }
}
```

```
int getSize(ArrayStack<int> as)
```

```
{
    int siz= 0;
    while(!as.isEmpty())
    {
        int v = 0;
        as.pop(v);
        siz++;
    }
    return siz;
}
```

```
int max(ArrayStack<int> as)
```

```
{
    int v = 0;
    as.pop(v);
    int mx = v;
    while(!as.isEmpty())
```

```

{
    as.pop(v);
    if(mx < v)
    {
        mx = v;
    }
}
}

```

// Write recursive code for the following function:
// the function calculates frequency of each value and return in a Stack
// You may create more helper functions but all functions must be recursive.
ArrayStack<int> frequency(ArrayStack<int> adt);

Task 4

In this task, your will write recursive functions on ArrayQueue. The iterative functions are given below:

```

void copyQueue(ArrayQueue<int> src, ArrayQueue<int> &copy)
{
    while(!src.isEmpty())
    {
        int v = src.dequeue();
        copy.enqueue(v);
    }
}

```

```

void display(ArrayQueue<int> as)
{
    while(!as.isEmpty())
    {
        int v = as.dequeue();
        cout << v << " ";
    }
}

```

```

int getSize(ArrayQueue<int> as)
{
    int size = 0;
    while(!as.isEmpty())
    {
        int v = as.dequeue();
        size++;
    }
    return size;
}

```

```

int max(ArrayQueue<int> as)
{
    mx = as.dequeue();
    while(!as.isEmpty())
    {

```

```

int v = as.dequeue();
if(mx < v)
{
    mx = v;
}
}
return v;
}

```

// Write recursive code for the following function:
// the function calculates frequency of each value and return in a ArrayQueue
// You may create more helper functions but all functions must be recursive.
ArrayQueue <int> frequency(ArrayQueue <int> adt);

Task 5

In this task, your will write recursive functions on DoublyLinkedList. The iterative functions are given below:

```

void LLCopy(ConcreteDoublyLinkedList<int> src, ConcreteDoublyLinkedList<int> &copy)
{
    while(!src.isEmpty())
    {
        int x = 0;
        src.removeFromLast(x);
        copy.insertAtFirst(x);
    }
}

void display(ConcreteDoublyLinkedList<int> as)
{
    while(!as.isEmpty())
    {
        int v = 0;
        as.removeFromFirst(v);
        cout << v << " -> ";
    }
}

int getSize(ConcreteDoublyLinkedList<int> as)
{
    int size = 0;
    while(!as.isEmpty())
    {
        int v = 0;
        as.removeFromLast(v);
    }
    size++;
}

int max(ConcreteDoublyLinkedList<int> as)
{

```

```
int mx = 0;
as.removeFromLast(mx);
while(!as.isEmpty())
{
    int v = 0;
    as.removeFromLast(v);
    if(mx < v)
    {
        mx = v;
    }
}
}
```

// Write recursive code for the following function:

// the function calculates frequency of each value and return in a ConcreteDoublyLinkedList

// You may create more helper functions but all functions must be recursive.

ConcreteDoublyLinkedList <int> frequency(ConcreteDoublyLinkedList <int> adt);